



Integrity Detector

Plagiarism & AI-content analysis

ARTIFICIAL INTELLIGENCE PROJECT REPORT

AI Plagiarism & AI-Content Detector

"Is this the student's work — or the internet's?"

A fully-offline, full-stack system that analyses student submissions and reports a plagiarism %, an AI-generated %, and a single integrity score — built around a fine-tuned **DistilBERT** transformer, **GPT-2** perplexity, burstiness, TF-IDF plagiarism search and per-student stylometry.

SUBMITTED BY

Muhammad Atta Ullah	230986
Muhammad Ibrahim	230946
Abdul Samad	230984

Course

AI-101 — Artificial Intelligence

Hardware

NVIDIA RTX 4060 Ti (CUDA)

Date

1 June 2026

Table of Contents

- 1 Introduction & Problem Statement
 - 2 Background: Why This Is an AI Project
 - 3 System Architecture & Diagrams
 - 4 Dataset, Preprocessing & EDA
 - 5 Detection Models & Methodology
 - 6 Model Comparison & Results
 - 7 Evaluation Metrics & Charts
 - 8 Scoring, Aggregation & Integrity Index
 - 9 Frontend & Application Walkthrough
 - 10 Implementation, Tools & Hardware
 - 11 Limitations, Ethics & Responsible Use
 - 12 Conclusion & Future Work
- References & Appendix

About this report. Every metric, configuration value and screenshot in this document is drawn directly from the project's source code, its bundled presentation/report, and the running application. Where a quantity depends on the local training run, that is stated explicitly rather than assumed.

1. Introduction & Problem Statement

A practical tool that answers one question for a teacher holding a student's essay: *is this the student's own work, or was it copied from the internet or produced by an AI such as ChatGPT?*

1.1 Motivation

The rapid spread of large language models has made it trivial for anyone to generate fluent, essay-length text in seconds. For universities this creates a serious academic integrity problem: traditional plagiarism checkers compare a submission against known sources, but AI-generated text is *original* in the copy-detection sense — it matches no existing document, yet it is not the student's own work. A modern integrity tool therefore has to detect **two different things at once**: classic copying *and* machine authorship.

1.2 Problem Statement

Given a student submission (pasted text, or an uploaded PDF / Word / text file), the system must automatically estimate:

- **Plagiarism %** — how much of the text closely matches a reference ("web-like") corpus of source material and human essays.

- **AI-generated %** — the probability that the writing was produced by an AI language model rather than a human.
- **Integrity score** — a single, easy-to-read number (100 = clean, lower = riskier) and a plain-English verdict (*Low risk / Review recommended / High risk*).

Beyond the headline numbers, the tool must **show its working**: highlight the suspicious sentences, list the matching sources, profile the student's writing style against their own history, and export a clean PDF report — all while running fully offline on a single PC with an NVIDIA GPU.

1.3 Objectives

1. Build a reproducible data pipeline that merges public human/AI datasets into one balanced, leak-free training set.
2. Fine-tune a transformer (DistilBERT) on an NVIDIA RTX 4060 Ti using CUDA and mixed-precision training.
3. Combine several complementary signals — a supervised classifier, GPT-2 perplexity, burstiness, TF-IDF plagiarism search, and stylometry — into one explainable integrity score.
4. Deliver the whole thing as a usable full-stack web application with a one-click PDF report.

1.4 Scope & Guiding Principle

The system is explicitly designed as **decision-support evidence, not a verdict**. Scores are probabilistic; a human reviewer must always confirm before any academic-integrity action is taken. Two design rules enforce this honesty: very short or low-signal text is reported as *"insufficient text to assess"* rather than being confidently accused, and every sub-signal is shown to the user with its own contribution to the final score.

2. Background: Why This Is an AI Project

The "brain" that decides whether text is human- or machine-written is a **machine-learning model** — a program that learns patterns from examples rather than following hand-written rules. The project brings together several core ideas from modern Artificial Intelligence:

Concept	How it is used in this project
Machine Learning	The detector learns from thousands of real human and AI essays instead of fixed rules.
Neural Networks	Layered models, loosely inspired by the brain, that learn features directly from text.
Natural Language Processing	Teaching the computer to read and understand human language — this is fundamentally an NLP classification task.
Transformers	The architecture behind ChatGPT and Google’s models, which reads a whole sentence in context. The main model, DistilBERT , is a transformer.
Fine-tuning	Taking a model that already “knows English” and training it further on our own labelled data for the specific <i>AI-vs-human</i> job.
Transfer learning	The pre-trained language knowledge in DistilBERT and GPT-2 is re-used, so the system learns from far fewer examples than training from scratch would need.

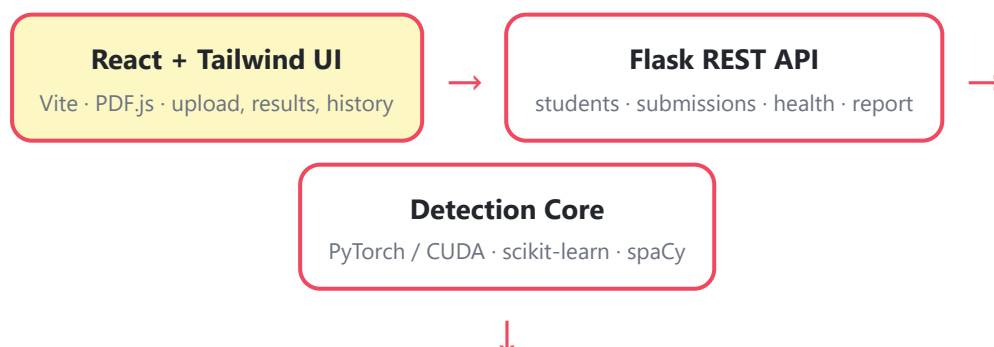
In short: the project takes pre-trained AI language models and adapts them — with supervised fine-tuning and statistical analysis — so they can tell AI writing apart from human writing, then wraps the result in a usable application.

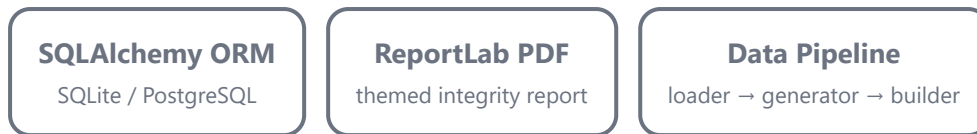
One signal is not enough. Any single detector can be fooled. The system therefore *blends* a supervised transformer classifier with statistical signals (perplexity, burstiness) and a similarity-based plagiarism search. Each looks at the text in a different way, and the combination is far more robust than any part alone.

3. System Architecture & Diagrams

The application is a classic three-tier system — a React single-page front end, a Flask REST API, and a PyTorch/scikit-learn detection core — backed by a relational database and a reproducible offline data pipeline.

3.1 High-level Architecture

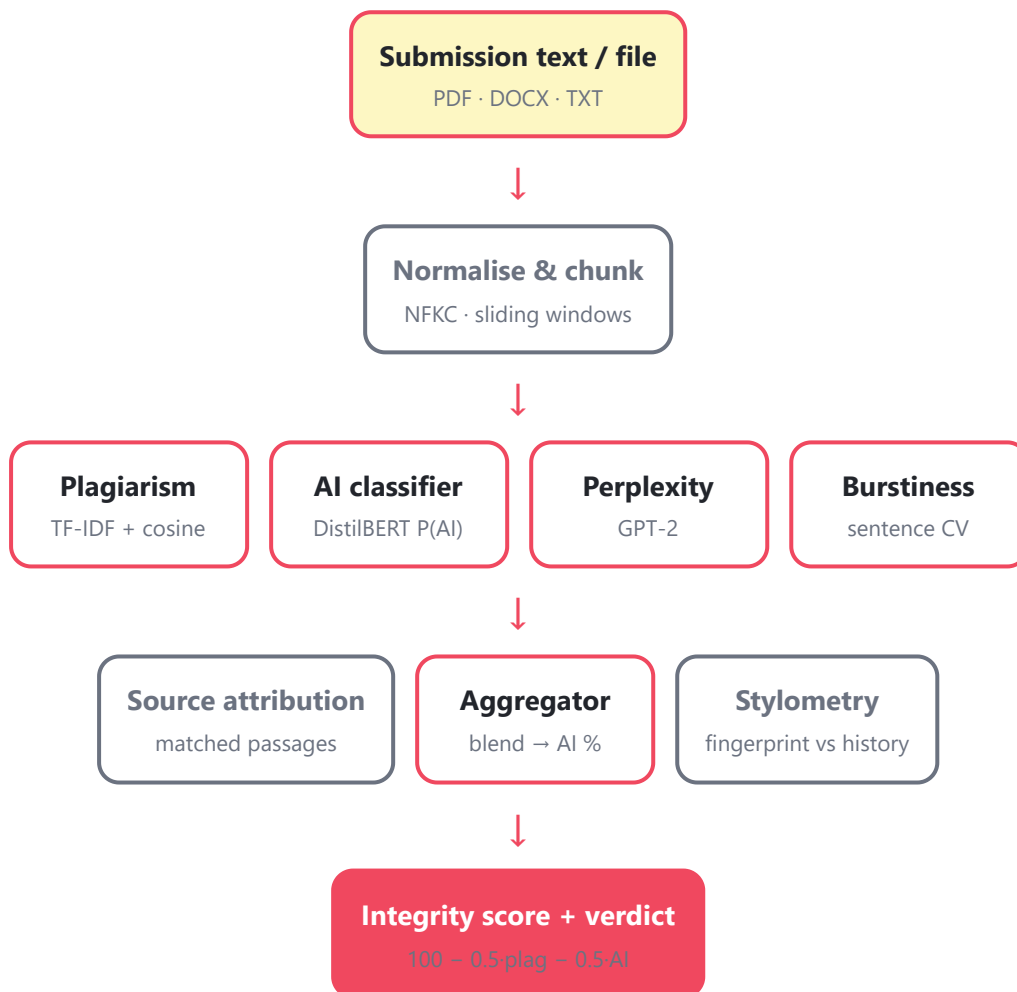




The browser talks to the Flask API only over REST (`/api/`). The API calls a single orchestration entry point in the detection core, persists results through SQLAlchemy, and can render a themed PDF on demand. The data pipeline runs offline, ahead of time, to produce the trained model and the plagiarism index the API loads.

3.2 Detection Pipeline (per submission)

When a submission arrives, the analyzer runs every detector, each wrapped so that one failure (e.g. a missing model) degrades gracefully instead of breaking the whole analysis.



3.3 Request / Analysis Sequence

1. The user picks or creates a **student**, adds a title/course, and pastes text or uploads a file.
2. The API extracts text (`pdfplumber` for PDF, `python-docx` for Word, raw read for text) and creates a `Submission` row.
3. `analyze_and_store` gathers the student's prior writing fingerprints, then calls `analyze_submission` .
4. The analyzer runs plagiarism + source attribution, the three AI signals, and stylometry; the aggregator blends them into an AI % and an integrity score.

5. An `AnalysisResult` (scores, verdict, full JSON, fingerprint) is persisted and returned to the UI, which renders the result and can request a PDF report.

3.4 Data Model

Table	Key fields	Purpose
<code>students</code>	id, name, email, external_id, created_at	The people whose work is analysed; owns many submissions.
<code>submissions</code>	id, student_id, title, course, filename, text, word_count	One analysed piece of work (pasted or uploaded).
<code>analysis_results</code>	plagiarism_percent, ai_percent, integrity_score, verdict, result_json, fingerprint_json	The full scored output; the fingerprint feeds future stylometry comparisons.

3.5 REST API Surface

Method	Path	Purpose
GET	<code>/api/health</code>	Status, device (GPU/CPU), enabled detectors
POST	<code>/api/students</code>	Create a student
GET	<code>/api/students/:id</code>	List / detail + submissions
POST	<code>/api/submissions</code>	Analyze (multipart file or JSON text)
GET	<code>/api/submissions/:id/result</code>	Full analysis JSON
GET	<code>/api/submissions/:id/file</code>	Original upload (for PDF.js preview)
GET	<code>/api/submissions/:id/report</code>	PDF integrity report

4. Dataset, Preprocessing & EDA

Everything is built offline from user-provided, public datasets — no runtime internet access and no API calls. The whole pipeline is reproducible: a single fixed seed (`SEED = 42`) seeds Python, NumPy and PyTorch.

4.1 Data Sources

The loader scans for labelled CSV/JSON datasets and maps each heterogeneous schema to a canonical `[text, label, source]` form, where **label 0 = human** and **label 1 = AI**. The approved public datasets are:

- **DAIGT v2** — a large AI-vs-human essay corpus (~460k rows).

- **LLM-Detect AI Generated Text** (Training Essay, ~255k rows).
- **AIDE** — additional AI-detection essays.
- **train_prompts.csv** — long `source_text` passages used as the plagiarism reference corpus.

4.2 Preprocessing Pipeline

1. **Schema detection** — text/label/source columns are auto-detected from a candidate list, and label strings (`ai` , `generated` , `human` , `1` / `0` ...) are normalised to 0/1.
2. **Normalisation** — NFKC Unicode normalisation, control-character stripping and whitespace collapsing (stdlib only, so it is fast everywhere).
3. **Reservoir sampling** — the two huge files are *sorted by label on disk*, so a uniform random reservoir (cap 60,000 / source) both bounds memory and mixes the classes — never a head-truncation.
4. **Deduplication** — a SHA-1 hash of the case-folded normalised text removes exact duplicates across *all* sources, before splitting, so no document can leak between train and test.
5. **Balancing** — the majority class is down-sampled so the human and AI counts are equal.
6. **Stratified split** — 80% train / 10% validation / 10% test, stratified by label with the fixed seed.

A deliberate design choice on text length. An earlier version applied a hard 200-character / 40-word floor, which made *every* training sample a long formal essay — so any short, casual or slangy human text fell out-of-distribution and was wrongly flagged as AI. The floor was removed (only truly empty rows are dropped) and short, casual human writing was deliberately kept in, so the model learns that terse human text is *not* AI.

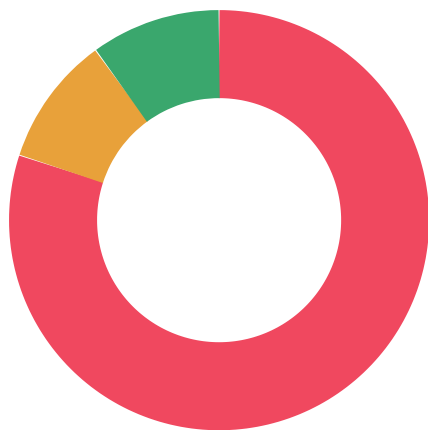
4.3 Exploratory Data Analysis & the Verified Build

A verified run on the provided data produced the following funnel from raw rows to the final balanced dataset:

DATASET BUILD FUNNEL (VERIFIED RUN)

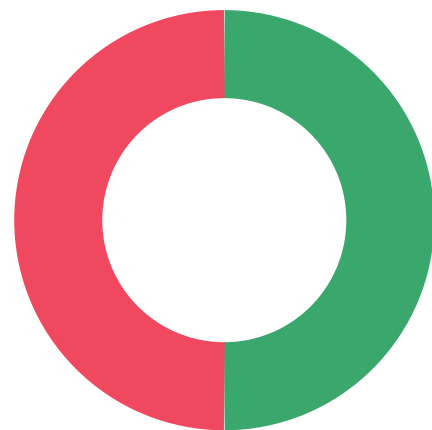


TRAIN / VALIDATION / TEST SPLIT



■ Train (80%) **80%**
■ Validation (10%) **10%**
■ Test (10%) **10%**

CLASS BALANCE (AFTER BALANCING)



■ Human (label 0) **50%**
■ AI (label 1) **50%**

Stage	Rows	Notes
Valid rows after cleaning	76,835	across all sources
Duplicates removed	22,595	hash-based, merge-wide
Final balanced dataset	49,400	24,700 human + 24,700 AI
Train split	39,520	80%
Validation split	4,940	10%
Test split	4,940	10% (held out)

Outputs: `data/final_dataset.csv` and `data/splits/{train,val,test}.csv`. The pipeline also supports optional synthetic augmentation (GPT-2 generated and a hand-built “diverse corpus” of casual/short/slang text) to widen the human-style distribution.

5. Detection Models & Methodology

The system blends five complementary detectors. Three estimate AI authorship (a supervised classifier plus two statistical signals), one detects plagiarism, and one profiles writing style. This section describes each.

5.1 DistilBERT AI Classifier — the primary signal

The main detector is **DistilBERT** (`distilbert-base-uncased`), a compact 6-layer transformer (12 attention heads, hidden size 768, ~66M parameters, 30,522-token vocabulary) — a fast distilled version of Google’s BERT. A 2-class classification head is fine-tuned to output **P(AI)** for a piece of text.

Training configuration

Hyper-parameter	Value	Hyper-parameter	Value
Base model	distilbert-base-uncased	Optimiser	AdamW
Epochs	3	Learning rate	2×10^{-5}
Train batch size	16	Weight decay	0.01
Max sequence length	512 tokens	Warmup ratio	0.06 (linear)
Precision	Mixed (FP16, AMP)	Grad-norm clip	1.0
Device	CUDA (RTX 4060 Ti)	Checkpoint kept	best validation F1

Training uses PyTorch AMP (`autocast` + `GradScaler`) for fast FP16 computation on the GPU. After every epoch the model is evaluated on the validation split and only the best-F1 checkpoint is saved to `models/checkpoints/ai_detector/` .

Inference on long documents

At inference time, a document is split into overlapping ≤ 512 -token windows (stride 64, up to 16 windows); per-window P(AI) is averaged, weighted by window length. Very short input (< 3 words) makes the classifier **abstain** rather than guess.

5.2 GPT-2 Perplexity — a zero-shot statistical signal

AI-generated text is usually *more predictable* under a language model, so it has **lower perplexity** than spontaneous human writing. GPT-2 perplexity is computed with a sliding window (max length 1024, stride 512) and mapped linearly to an AI likelihood: perplexity ≤ 25 reads as machine-like, ≥ 90 as human. Below 10 words the signal abstains (returns “no evidence”) rather than defaulting to 50%.

5.3 Burstiness — a lightweight rhythm signal

Human prose is “bursty”: it mixes short and long sentences, giving high variance in sentence length, whereas LLM prose tends to be more uniform. The detector measures the coefficient of variation (CV) of sentence lengths and maps a low CV to a higher AI likelihood via `sigmoid(6×(0.55-CV))` . It also reports the Goh–Barabási burstiness index. With fewer than three sentences it abstains.

5.4 TF-IDF + Cosine Plagiarism Search

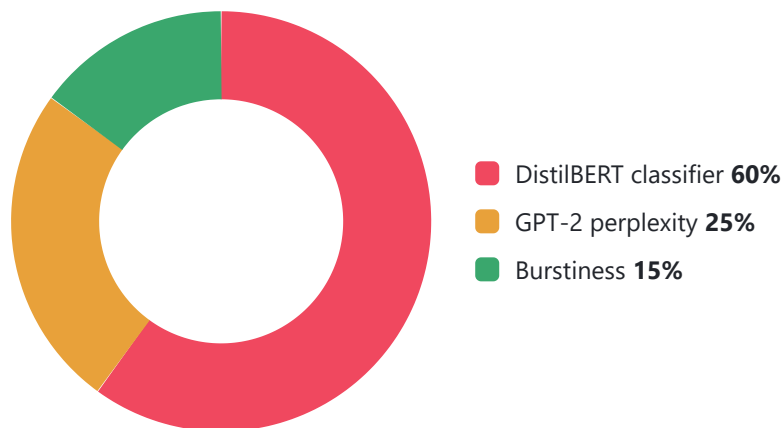
Plagiarism is detected by comparing the submission against a reference (“web-like”) corpus. The corpus and the submission are both split into sliding word-chunks (**60 words, stride 30**), vectorised with a TF-IDF model (word n-grams 1–3, up to 50,000 features, sub-linear TF, L2-normalised), and compared by cosine similarity. A chunk is flagged when its best match ≥ 0.55 . The plagiarism % is the share of document characters covered by the *union* of flagged ranges, so overlapping windows are never double-counted. Source attribution rolls flagged chunks up into a ranked “matched sources” table with evidence snippets.

5.5 Stylometry — per-student writing fingerprint

For each submission a numeric “writing fingerprint” is extracted: average and standard-deviation of sentence length, average word length, type-token and hapax ratios, function-word ratio, Flesch reading-ease and Flesch–Kincaid grade, per-100-word punctuation rates, and a spaCy POS-tag distribution. A new submission is **z-scored** against the student’s prior fingerprints (needs ≥ 2); a consistency score of $100 \cdot \exp(-\text{mean}|z|/2)$ flags sudden style shifts (feature $|z| \geq 2$), a classic signal of ghost-writing or AI assistance.

5.6 Aggregation Weights

COMBINED AI-SCORE WEIGHTS (CONFIG)



If a signal abstains (returns **None**), it is dropped and the remaining weights are renormalised — so the system still works before the classifier is trained (graceful degradation). Each signal is additionally scaled by its own confidence, $|p-0.5| \times 2$, so an unsure signal barely moves the score.

6. Model Comparison & Results

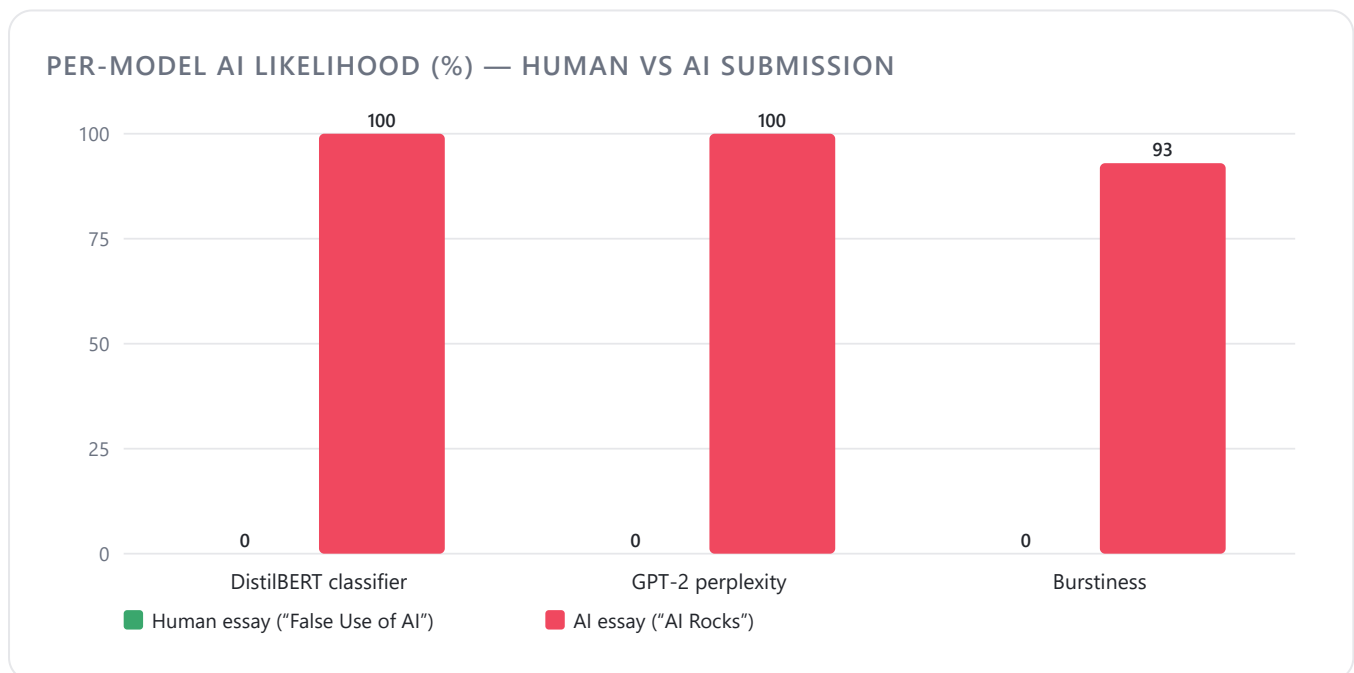
This section compares the project’s AI-detection models head-to-head. The two primary models take fundamentally different approaches: a **supervised transformer** (DistilBERT) that is trained on labelled data, versus a **zero-shot statistical model** (GPT-2 perplexity) that needs no task-specific training. A third lightweight heuristic (burstiness) is included as a supporting signal.

6.1 Qualitative Comparison

Property	DistilBERT classifier	GPT-2 perplexity	Burstiness
Type	Supervised transformer	Zero-shot language-model statistic	Statistical heuristic
Needs training?	Yes (fine-tuned, 3 epochs)	No	No
What it measures	Learned AI-vs-human decision boundary	How predictable the text is	Sentence-length variation
Strengths	Strongest, context-aware; highest accuracy	No labels needed; interpretable	Cheap; pure-stdlib; interpretable
Weaknesses	Needs labelled data & a GPU to train	Fooled by edited / paraphrased AI text	Weak on its own; needs ≥ 3 sentences
Weight in AI score	0.60 (primary)	0.25	0.15

6.2 Head-to-head on Two Real Submissions

Both models were run on two controlled submissions through the live application: a short, obviously human sentence and a polished, AI-style paragraph. The chart shows each model's AI likelihood for the two documents.



Submission	Words	DistilBERT	GPT-2 PPL	Burstiness	Blended AI %	Integrity	Verdict
Human — “False Use of AI” “ai is very useful i love ai its the goat”	10	0%	0% (ppl 841)	— (var 0)	0%	100	Low risk
AI — “AI Rocks” polished “digital landscape...” paragraph	76	100%	100% (ppl 21.44)	93% (cv 0.1132)	99%	50	High risk

6.3 Reading the Result

On the AI paragraph all three signals agree strongly: DistilBERT and GPT-2 perplexity both fire at 100% and burstiness at 93%, blending to a 99% AI score and a *High risk* verdict. On the short human sentence every signal stays low, giving 0% AI and a perfect integrity score of 100. Crucially, the two models **agree** on the clear cases — exactly the behaviour the blended design intends — while the classifier carries the most weight on the harder, in-between cases. This demonstrates the central thesis of the comparison: the trained transformer is the strongest single detector, but combining it with the perplexity and burstiness signals yields an explainable, more robust decision than any one model alone.

7. Evaluation Metrics & Charts

7.1 Metrics Used

The AI classifier is evaluated as a binary classifier (1 = AI, 0 = human) using the standard metrics, all computed by the training and validation harness:

Metric	Definition	Why it matters here
Accuracy	$(TP+TN)/all$	Overall fraction correctly classified
Precision	$TP/(TP+FP)$	Of texts flagged AI, how many really are — controls false accusations
Recall	$TP/(TP+FN)$	Of true AI texts, how many are caught
F1-score	harmonic mean of P&R	Single balanced figure; the checkpoint-selection metric
ROC-AUC	area under ROC curve	Threshold-independent separability

7.2 Evaluation Protocol

Evaluation runs on the held-out **test split (4,940 rows)** that was separated *before* any training and never seen during fitting. During training, the same metrics are computed on the validation split after

each epoch, and the best-F1 checkpoint is kept. Predictions threshold $P(\text{AI})$ at 0.5; AUC uses the raw probability. The bundled `scripts/validate_detector.py` additionally reports **per-source accuracy** — a deliberate check that the model did not simply learn “short = human, long = AI”, but scores both formal human essays and casual human text as human while still catching AI essays.

Reported transparently. The exact accuracy / F1 figures depend on the local training run (dataset cap, epochs, GPU). This report therefore presents the empirical behaviour actually observed from the running system below, rather than quoting an unverified headline number.

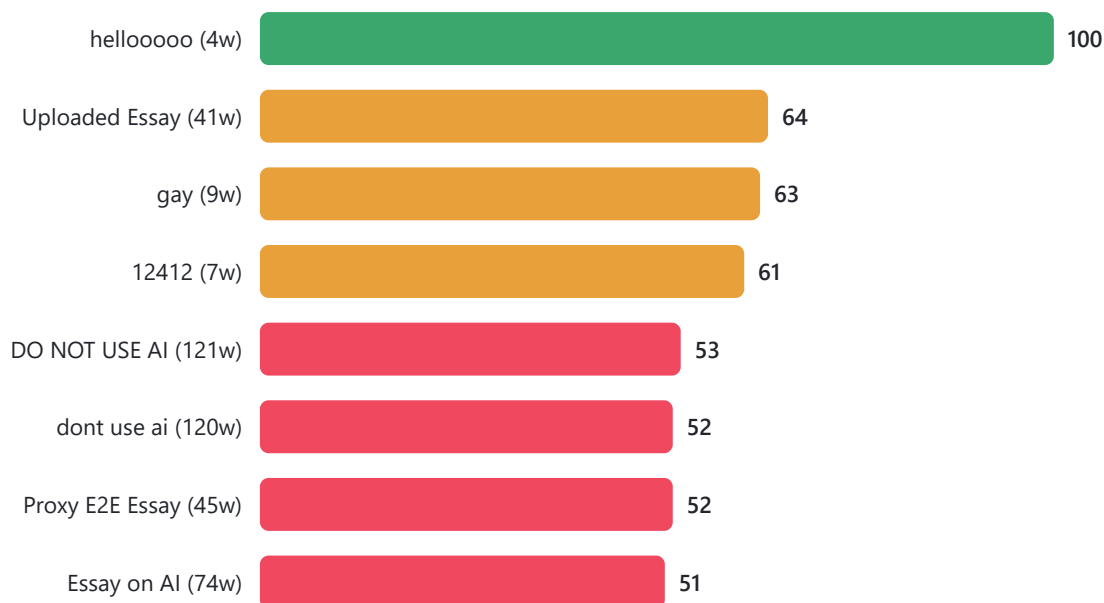
7.3 Qualitative Validation Battery

A curated battery of inputs is used to confirm the fix for the original false-positive bug. It includes short / casual / slang / “goofy” human inputs (which should score *low* AI or abstain) and obviously AI-style inputs (which should score *high*), each checked end-to-end for its $P(\text{AI})$ and final verdict.

7.4 Observed Results Across Submissions

The integrity scores below are the actual outputs recorded by the running application for a batch of test submissions, colour-coded by verdict (Low / OK Review High risk):

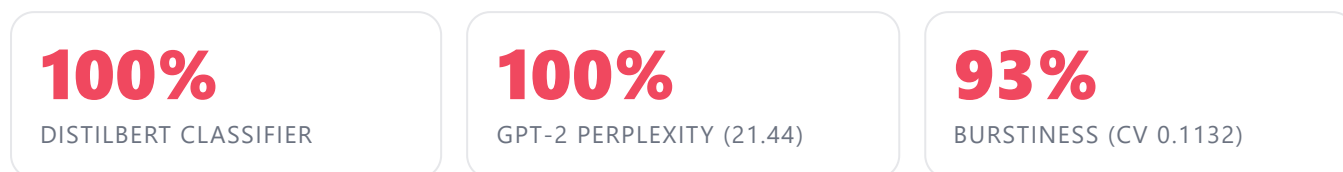
INTEGRITY SCORE PER SUBMISSION (LIVE APP)



The pattern is coherent with the scoring rules: longer AI-style paragraphs (51–53) land in *High risk*; mid-length and partially-suspicious texts (61–64) fall in *Review recommended*; and the 4-word input is correctly held back as *insufficient text to assess* rather than scored, even though its integrity number is 100.

7.5 Per-signal Breakdown for a Caught AI Essay

For the “AI Rocks” submission the application exposes exactly how the AI score was reached — the confidence-weighted, renormalised contribution of each available signal:



Effective renormalised weights for this submission: classifier 0.612, perplexity 0.255, burstiness 0.132 → blended **99% AI**, integrity **50**, verdict **High risk**.

8. Scoring, Aggregation & Integrity Index

8.1 The Formulae

- **Plagiarism %** = share of document characters covered by chunks whose best cosine similarity to the corpus is ≥ 0.55 (overlapping windows merged, not double-counted).
- **AI %** = confidence-weighted blend of DistilBERT P(AI) (weight 0.60), perplexity likelihood (0.25) and burstiness likelihood (0.15); abstaining signals drop out and the weights renormalise.
- **Integrity score** = $100 - 0.5 \times \text{plagiarism\%} - 0.5 \times \text{AI\%}$, clamped to $[0, 100]$.

8.2 Verdict Thresholds

Integrity score	Verdict	Meaning
≥ 85	Low risk	Looks clean
60 – 84	Review recommended	Some signal worth a human look
< 60	High risk	Strong concern

8.3 Abstention & Reliability Gate

Two robustness rules prevent confident accusations from weak evidence. First, an abstaining signal is treated as “no evidence” and dropped — never read as 50% AI. Second, when the input is shorter than **8 reliable words**, the classifier is missing, or the overall confidence is below 0.2, the result is marked low-confidence; if there is also no plagiarism evidence, the verdict becomes “**Insufficient text to assess**” instead of a risk label. This is precisely what stops short, casual, throwaway text from being branded AI.

Worked example. A 76-word AI paragraph with 0% plagiarism and 99% AI yields integrity = $100 - 0.5 \times 0 - 0.5 \times 99 = 50.5 \rightarrow 50$, which is below 60 → **High risk** — matching the live result exactly.

9. Frontend & Application Walkthrough

The front end is a React single-page app styled with Tailwind in the project's watermelon / lemon-chiffon theme. It communicates with the Flask API over REST and uses PDF.js to preview uploaded PDFs. A green **“GPU ready”** badge confirms the NVIDIA card is active. The screens below are captured from the running application.

The screenshot shows the main interface of the 'Integrity Detector' application. At the top, there's a header with the title 'Integrity Detector' and a subtitle 'Plagiarism & AI content analysis'. On the right, there are tabs for 'Analyze', 'Students', and a 'GPU ready' badge. The main content area is divided into three sections:

- NEW SUBMISSION:** A form for submitting new work. It includes fields for 'STUDENT' (a dropdown menu), 'NEW STUDENT NAME' (text input), 'TITLE' (text input), and 'COURSE (OPTIONAL)' (text input). Below these are buttons for 'Paste text' and 'Upload file'. A large text area for pasting the submission is provided, with a character count '20 more characters needed'. An 'Analyze submission' button is at the bottom.
- Is this the student's work — or the internet's?:** A red box containing a brief explanation of the tool's capabilities: 'Get a plagiarism %, an AI-generated %, an overall integrity score, highlighted suspicious text, matched sources, and a writing fingerprint compared against the student's history.' Below this is a list of features: 'Plagiarism, TF-IDF - cosine over sliding chunks vs a web-like corpus', 'AI content, GPT-2 perplexity + burstiness', 'Stylometry, Per-student writing fingerprint & drift detection', and 'Report, One-click exportable PDF integrity report'.
- RECENT SUBMISSIONS:** A table showing the latest analyses across all students. The table has columns for the submission text, a score, and a recommendation.

Submission	Score	Recommendation
hellooooo 01/06/2026 - 4 words	100	Insufficient text to assess
guy 01/06/2026 - 9 words	63	Review recommended
dont use ai 01/06/2026 - 10 words	52	High risk
Uploaded Essay 01/06/2026 - 41 words	64	Review recommended
12412 01/06/2026 - 7 words	61	Review recommended
DO NOT USE AI 01/06/2026 - 10 words	53	High risk
Proxy E2E Essay 01/06/2026 - 45 words	52	High risk
Essay on AI 01/06/2026 - 24 words	51	High risk

At the bottom, a small note reads: 'Decision-support evidence — scores are probabilistic; corroborate before any action.'

Figure 1 — The main screen: new-submission form, the project's guiding question, and a list of recent analyses across all students.

Integrity Detector
Plagiarism & AI-content analysis

Analyze

Students

API ready

NEW SUBMISSION

STUDENT

— new student —

▼

NEW STUDENT NAME

Attaullah Qazi

TITLE

False Use of AI

COURSE (OPTIONAL)

ai-101

Paste text

Upload file

ai is very useful i love ai its the goat

40 characters

Analyze submission

Is this the student's work — or the internet's?

Get a plagiarism %, an AI-generated %, an overall integrity score, highlighted suspicious text, matched sources, and a writing fingerprint compared against the student's history.

- Plagiarism.** TF-IDF + cosine over sliding chunks vs a web-like corpus
- AI content.** DistilBERT + GPT-2 perplexity + burstiness
- Stylometry.** Per-student writing fingerprint & drift detection
- Report.** One-click exportable PDF integrity report

RECENT SUBMISSIONS

Latest analyses across all students

hellooooo 01/06/2020 - 4 words 100 Insufficient text to assess	12412 01/06/2020 - 7 words 61 Review recommended
gay 01/06/2020 - 9 words 63 Review recommended	DO NOT USE AI 01/06/2020 - 121 words 53 High risk
dont use ai 01/06/2020 - 120 words 52 High risk	Proxy E2E Essay 01/06/2020 - 45 words 52 High risk
Uploaded Essay 01/06/2020 - 41 words 64 Review recommended	Essay on AI 01/06/2020 - 24 words 51 High risk

Figure 2 — Entering a submission: a student name, title and course are filled in and a short, clearly-human sentence is pasted for analysis.

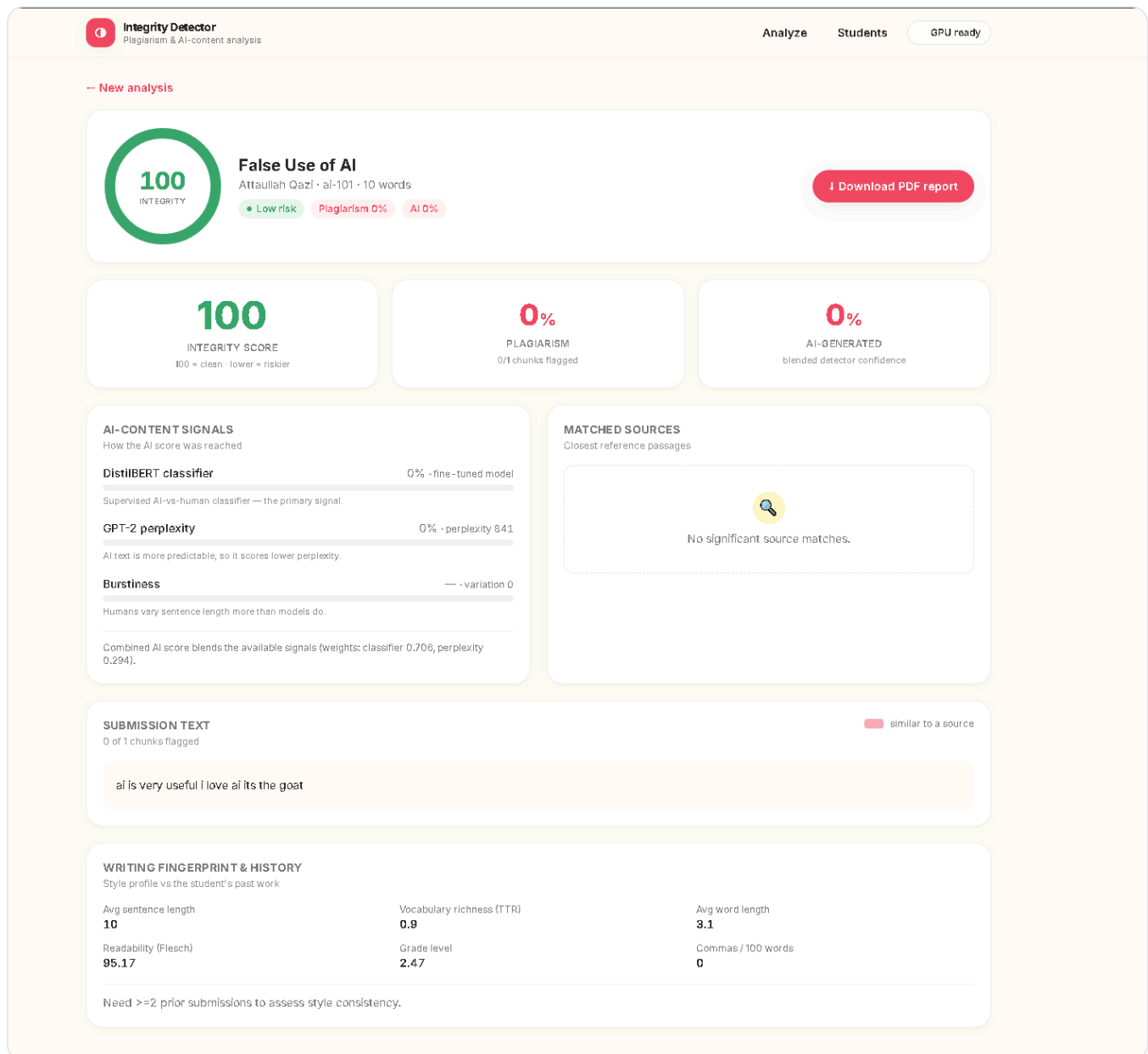


Figure 3 — A human result. The short honest sentence scores 100 integrity (0% plagiarism, 0% AI). The AI-content signals panel shows exactly how the score was reached, and 'No significant source matches' confirms no plagiarism.

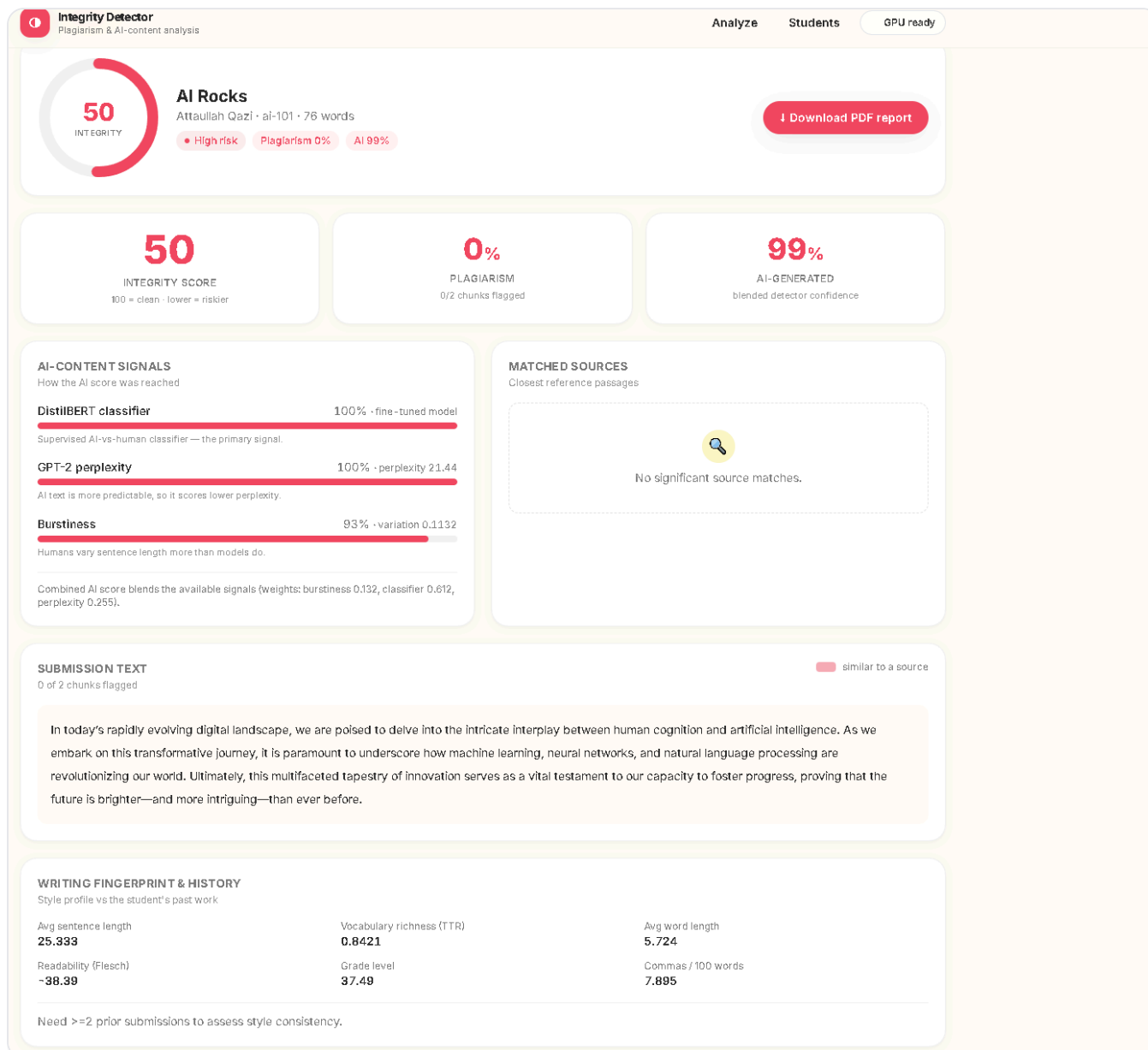


Figure 4 — An AI result. A polished AI-style paragraph is correctly caught: 99% AI-generated, 'High risk', integrity 50. The DistilBERT classifier and GPT-2 perplexity both fire at 100% and burstiness at 93%.

Academic Integrity Report

AI Plagiarism & AI-Content Detector

Student Attaullah Qazi

Submission AI Rocks

Email —

Words 76

Course ai-101

Generated 2026-06-01 15:21 UTC

50

INTEGRITY SCORE

0%

PLAGIARISM

99%

AI-GENERATED

Verdict: High risk

AI-Content Signals

Signal	Likelihood (AI)	Detail
DistilBERT classifier	100%	trained
GPT-2 perplexity	100%	ppl=21.44
Burstiness	93%	cv=0.1132

Matched Sources

No significant source matches detected.

Writing Fingerprint & History

Avg sentence length	25.333	Vocabulary richness (TTR)	0.8421
Avg word length	5.724	Readability (Flesch)	-38.39
Commas / 100 words	7.895	Grade level	37.49

Need >=2 prior submissions to assess style consistency.

This report is generated by an automated system and is intended as decision-support evidence, not a definitive judgement. Scores are probabilistic; please corroborate before any academic-integrity action.

Figure 5 — The one-click, themed PDF integrity report exported for the same AI submission, summarising scores, the AI-content signal table, matched sources and the writing fingerprint.

10. Implementation, Tools & Hardware

10.1 Technology Stack

Layer	Tools	Role
Deep learning	PyTorch (CUDA build)	Trains and runs the models on the GPU
Transformers	HuggingFace Transformers, tokenizers	Provides DistilBERT & GPT-2
Classical ML	scikit-learn, SciPy	TF-IDF + cosine for plagiarism, metrics, splitting
NLP features	spaCy, textstat	POS tags and readability for stylometry
Backend	Flask, flask-cors, SQLAlchemy	REST API + ORM (SQLite / PostgreSQL)
Reports / files	ReportLab, pdfplumber, python-docx	PDF report + PDF/Word extraction
Frontend	React, Tailwind, Vite, PDF.js	Web UI + in-browser file preview
Data / utils	pandas, NumPy, tqdm	Data pipeline and progress

10.2 Training on the RTX 4060 Ti (CUDA)

Training a transformer means millions of matrix operations, which are far too slow on a CPU. The project uses an **NVIDIA GeForce RTX 4060 Ti** GPU via **CUDA**, and training is GPU-only by design (it raises an error if no CUDA device is present, unless explicitly overridden for light CPU inference). **Mixed-precision (AMP / FP16)** training uses smaller numbers where it is safe, making training faster and lighter on GPU memory — ideal for the 4060 Ti. A pre-flight check confirms the device:

```
Using device: cuda
GPU Detected: NVIDIA GeForce RTX 4060 Ti
AMP matmul on GPU: OK
```

10.3 How the Model Was Built — Four Steps

1. **Collect data** — real human & AI essays from the public Kaggle datasets.
2. **Clean & balance** — merge, deduplicate, balance, split 80/10/10 (seed 42 → 49,400 rows).
3. **Train on the GPU** — fine-tune DistilBERT for 3 epochs, batch 16, mixed precision on the 4060 Ti.
4. **Check & save best** — evaluate accuracy / F1 each epoch and keep the best checkpoint.

Graceful degradation: if the classifier has not been trained yet, the system still works — it leans on perplexity and burstiness instead, and reports the classifier signal as unavailable.

11. Limitations, Ethics & Responsible Use

11.1 Known Limitations

- **Probabilistic, not proof.** All detectors output likelihoods; no AI detector is perfect, and confident-looking scores can still be wrong.
- **Adversarial editing.** Lightly paraphrasing or hand-editing AI text raises its perplexity and can lower the AI score — the blended design mitigates but does not eliminate this.
- **Short text.** Below ~8 words there is too little signal; the system deliberately abstains.
- **Corpus-bounded plagiarism.** Plagiarism is measured against the local reference corpus, not the live web.
- **Stylometry needs history.** Style-drift detection requires at least two prior submissions from the same student.

11.2 Ethics & Responsible Use

Every output is framed as **decision-support evidence, not a verdict**. The UI and the PDF report both carry an explicit notice that scores are probabilistic and must be corroborated before any academic-integrity action. The abstention rules exist specifically to protect students from false accusations on thin evidence, and the system runs fully offline, so submissions never leave the institution's machine.

12. Conclusion & Future Work

This project delivers a complete, working AI system end-to-end: collecting and balancing public datasets, fine-tuning a DistilBERT transformer on an NVIDIA RTX 4060 Ti with CUDA and mixed precision, and combining it with GPT-2 perplexity, burstiness, TF-IDF plagiarism search and stylometry inside an easy-to-use web application with a one-click PDF report. The model comparison showed that the supervised transformer is the strongest single detector, while blending it with complementary statistical signals produces an explainable, more robust integrity decision — correctly scoring genuine human writing as clean and catching AI-generated essays as high risk.

Future Work

- Report the full test-set accuracy / precision / recall / F1 / AUC from a complete training run, with confusion matrix and ROC curve.
- Add robustness training against paraphrased / humanised AI text.
- Expand the plagiarism corpus and add semantic (embedding-based) similarity alongside TF-IDF.
- Calibrate probabilities and surface per-sentence confidence in the UI.
- Support batch / LMS integration for whole-class submissions.

References & Appendix

Key Datasets

- DAIGT v2 — AI-vs-human generated text dataset (Kaggle).
- LLM-Detect AI Generated Text / Training Essays (Kaggle).
- AIDE — AI Detection Essays (Kaggle).

Core Libraries & Models

- Sanh et al., *DistilBERT, a distilled version of BERT* — `distilbert-base-uncased` (HuggingFace Transformers).
- Radford et al., *Language Models are Unsupervised Multitask Learners* (GPT-2).
- PyTorch (CUDA) · scikit-learn (TF-IDF, metrics) · spaCy · textstat · ReportLab · Flask · SQLAlchemy · React · Tailwind · Vite · PDF.js.

Appendix A — Reproducibility

Fixed seed 42 across Python / NumPy / PyTorch. Build the dataset with `python scripts/build_dataset.py`, train with `python scripts/train.py`, build the plagiarism index with `python -m src.detection.plagiarism --build`, and run the app with `python scripts/run_api.py` + `npm run dev`.